

Mid 2

Question 1: Identify the correct React Hook. Provide a small code snippet demonstrating its usage

1. Fetch data from an API when the component mounts

Hook: `useEffect`

```
import { useEffect, useState } from "react";

function UserList() {
  const [users, setUsers] = useState([]);

  useEffect(() => {
    fetch("https://api.example.com/users")
      .then(res => res.json())
      .then(data => setUsers(data));
  }, []); // Empty dependency array = run once on mount

  return (
    <ul>
      {users.map(user => (
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
  );
}
```

2. Store a reference to a DOM element to focus an input on page load

Hook: `useRef`

```
import { useEffect, useRef } from "react";

function SearchBox() {
  const inputRef = useRef(null);
```

```

useEffect(() => {
    inputRef.current.focus();
}, []);

return <input type="text" ref={inputRef} placeholder="Search..."
/>;
}

```

3. Store a user's input in a form and update the UI as they type

Hook: useState

```

import { useState } from "react";

function NameForm() {
    const [name, setName] = useState("");
    return (
        <div>
            <input
                type="text"
                value={name}
                onChange={e => setName(e.target.value)}
                placeholder="Enter your name"
            />
            <p>Hello, {name}!</p>
        </div>
    );
}

```

Question 2:

In JSX, a return must contain a single parent element. Currently, you are trying to return two sibling elements (<h1> and <p>) directly.

Question 3:

// cartSlice.js

```
import { createSlice } from "@reduxjs/toolkit";

const cartSlice = createSlice({
  name: "cart",
  initialState: [],
  reducers: {
    addToCart: (state, action) => {
      state.push(action.payload);
    },
    removeFromCart: (state, action) => {
      return state.filter((item) => item.id !== action.payload);
    },
  },
});

export const { addToCart, removeFromCart } = cartSlice.actions;
export default cartSlice.reducer;
```

// product.js

```
export const products = [
  { id: 1, name: "Laptop", price: 1200 },
  { id: 2, name: "Phone", price: 800 },
  { id: 3, name: "Headphone", price: 200 },
];
```

// ProductList.js

```
import React from "react";
import { useDispatch } from "react-redux";
import { addToCart } from "../cartSlice";
```

```

import { products } from "../product";

const ProductList = () => {
  const dispatch = useDispatch();
  return (
    <div>
      <h2>Products</h2>
      {products.map((product) => (
        <div key={product.id}>
          <p>
            {product.name} - ${product.price}
          </p>
          <button onClick={() => dispatch(addToCart(product))}>Add
to Cart</button>
        </div>
      ))}
    </div>
  );
};

```

```

export default ProductList;

```

// CartPage.js

```

import React from "react";
import { useSelector, useDispatch } from "react-redux";
import { removeFromCart } from "../cartSlice";
const CartPage = () => {
  const cart = useSelector((state) => state.cart);
  const dispatch = useDispatch();
  return (

```

```

    <div>

    <h2>Cart</h2>

    {cart.length === 0 ? (
    <p>Cart is empty</p>
    ) : (
    cart.map((item) => (
        <div key={item.id}>

        <p>

        {item.name} - ${item.price}

        </p>

        <button onClick={() =>
dispatch(removeFromCart(item.id))}>

        Remove from Cart

        </button>

        </div>

    ))
    )}

    </div>

);

};

export default CartPage;

// App.js

import React from "react";

import { BrowserRouter as Router, Routes, Route, Link } from
"react-router-dom";

import { Provider } from "react-redux";

import { configureStore } from "@reduxjs/toolkit";

```

```
import cartReducer from "../cartSlice";
import ProductList from "../ProductList";
import CartPage from "../CartPage";

const store = configureStore({
  reducer: {
    cart: cartReducer,
  },
});

const App = () => {
  return (
    <Provider store={store}>
      <Router>
        <nav>
          <Link to="/">Home</Link> | <Link to="/cart">Cart</Link>
        </nav>
        <Routes>
          <Route path="/" element={<ProductList />} />
          <Route path="/cart" element={<CartPage />} />
        </Routes>
      </Router>
    </Provider>
  );
};

export default App;
```